



区块链原理、设计与应用实验报告

以太坊实验

姓 名：田佳铭
学 号：22920172204211
院 系：信息学院人工智能系
专 业：智能科学与技术
年 级：2017 级
指导教师：张哲民

2020 年 6 月 21 日

一、实验目的

本课程要求学生完成搭建以太坊的私有链并在私有链上部署自己设计的智能合约。学生需提交相关代码与实验报告，并在实验报告中详细描述提交代码的运行测试过程以便老师复现其功能。

二、前期准备

因为路径不同，有些代码需要进行修改。

在部署智能合约之前，需要做以下准备，主要是相关工具的准备和搭建自己的私有链。

(1) 如果没有 docker，需要运行 startDocker.sh 前面被注释掉的代码，在本机上安装 docker 和 ubuntu16:04 的镜像

(2) 安装完 docker 后，运行 startDocker.sh 中没有被注释的部分，启动 docker，注意这时候需要改一个路径，如图所示：

```
docker run -it --name ethereum \
-v /home/hxxz/work3:/home/work \
-p 8545:8545 -p 30303:30303 \
ubuntu:16.04
```

图 2-1 需要修改的路径

此路径为“代码”文件夹所在的路径，即，此路径中包含“”和“”两个子文件夹，后一个路径尽量不要修改，否则文件中很多

(3) 在 docker 的 /home/work/ethereumDemo 目录下运行 setupDocker.sh（可能需要先给与权限）。其中，在 ethereumDemo/ETH-demo 文件夹下如果没有 node_modules 文件夹的话，需要取消 setupDocker.sh 的最后几行（也可以将安装好的 node_modules 文件夹复制到此目录下）：

```
#setup web3
#cd ~
#npm install web3
#cp -r /root/node_modules /home/work/ethereumDemo/ETH-demo/node_modules
```

图 2-2 使用 npm 安装 web3

来安装 web3。

(4) 在 docker 的 /home/work/ethereumDemo/ETH-demo 中先运

行./bootnode.sh, 之后运行./start.sh ‘参数’即可以运行成功, 建议参数为两位数字, 例如 00, 因为这也参与端口号的构成。这样的话就可以搭建好自己的私链, 可以在其中进行新建账户并挖矿的操作。

在 start.sh 中如下部分:

```
# 如果启动前需要使用创世块初始化
if [ ! -d "$DIRECTORY" ]; then
    if [ ! -f $datadir/genesis ];then
        echo '{"config": {"chainId": 15, "homesteadBlock": 0, "eip150Block": 0,
"eip155Block": 0, "eip158Block": 0 }, "coinbase" :
"0x000000000000000000000000000000000000000000000000", "difficulty" : "0x40000",
"extraData" :
"0x04c13046fefc255d2ce8cb201b1646af093d3423ed281423d7897d68967aac75677a5f2d9f9a5
"gasLimit" : "0xffffffff", "nonce" : "0x0000000000000042", "mixhash" :
"0x0000000000000000000000000000000000000000000000000000000000000000",
"parentHash" :
"0x0000000000000000000000000000000000000000000000000000000000000000",
"timestamp" : "0x00", "alloc": { } }' > $datadir/genesis
```

图 2-3 写入创世区块部分

如果版本有改动或者需要新的创世区块格式, 可以修改粉红色的部分, 粉红色部分在此目录下的 genesis 文件中有备份。

备注: 理论上通过上述步骤就可以搭建成功私链并且安装成功需要的环境, 可以使用 solcjs 等来测试相关工具是否安装成功, 这其中可能会有路径的问题, 不过问题应该不大。

三、部署智能合约说明

可能是因为版本问题, 在我本地部署智能合约时, 使用的语法和老师的有所不同, 为了安全起见, 这里也仿照老师课堂的写法写了一些版本, 在相应的.js 文件的注释中, 但是因为不能在本地尝试, 所以不敢保证是对的。

3.1 前期需要的准备工作

前期需要对.js 文件进行一定的修改, 例如:

```
const HttpProvider="http://172.17.0.2:8100";
const HttpProvider1="http://172.17.0.2:8102";
```

图 3-1 .js 修改

这样的变量需要需改为"http:"+IP+端口号。其中, 端口号为 81+./start.sh ‘参数’中的参数。IP 地址可以通过 cat /etc/hosts 指令来获取。

如果需要重新生成智能合约的 bin 和 abi，可以使用 solcjs 相关命令，如果 bin 和 abi 的文件名有所改变，需要更改如图所示的路径：

```
var helloAbiBuf = fs.readFileSync("./com1_sol_People.abi");
var helloAbi = JSON.parse(Buffer.from(helloAbiBuf).toString());

var helloBinBuf = fs.readFileSync("./com1_sol_People.bin");
var helloBin = "0x" + Buffer.from(helloBinBuf).toString();
```

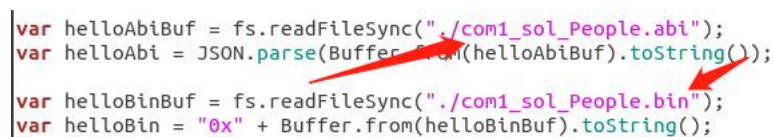


图 3-2 更改路径

password 需要修改为自己创建账户时使用的密码：

```
password="hsxz";
```

图 3-3 账户密码

在实际使用中，可能还有其他地方需要修改，这里无法全部考虑到。

3.2 智能合约说明

为了保险起见，这里写了几个智能合约。

3.2.1 test.sol

这是测试用的智能合约，主要的目的是测试智能合约是否可以在本地部署。相关的文件有：

- (1) test.sol：智能合约的原始文件
- (2) test_sol_People.bin：编译 test.sol 生成的 bin 文件
- (3) test_sol_People.abi：编译 test.sol 生成的 abi 文件
- (4) test.js：部署智能合约的文件

需要先运行 ./start.sh 00 和 ./start.sh 02（也可以根据需要进行修改、增删等），并且在 ./start.sh 00 中进行挖矿并有足够的钱（部署智能合约需要钱）。在本目录下运行 nodejs test.js（或者 node，根据本地的需要）（如上文所说，因为安装工具的版本不同等问题，可能需要用注释掉的那部分代码来进行部署）。并且立即启动挖矿，需要在 750s 内挖出第一个矿，并且再挖出一个矿后，测试成功。部署成功应该有回调，如果回调地址不为 null 的话，就可以获得合约地址，进行下一步的操作。（以上是未注释的代码的运行结果，注释代码部分的结果应该和课堂展示相似）

未注释部分 test.js 做了以下的一些事情：

- (1) 初始化操作，包括申请 web3、获取 bin 和 abi 的内容等
- (2) 创建合约模板
- (3) 获取 coinbase，以此来部署合约
- (4) 评估交易手续费
- (5) 解锁账户
- (6) 将交易发送到节点
- (7) 第一次回调，理论上就可以获得合约地址，发生事件 newname 后，调用 rename 函数，再次今次那个 (4) (5) (6) 步骤
- (8) 得到最终回调并显示出来

test.sol 的功能比较简单，和课堂上演示的 hello 效果基本相同，只不过是增加了一个事件——event newname(string);

运行 test.js 后，如图，可以看到在 geth 控制台，我们发现一个合约需要打包：

```
> txpool.status INFO [06-19|14:31:25.650] Submitted contract creation
    fullhash=0x2df020911113723f229303752d0db85089971c5e4228e7266bfa5983c7f7a15a
contract=0x3e62B66C4c9f4A8c94B829844157B13A6eb34Bf6
> miner.start()
INFO [06-19|14:31:27.030] Updated mining threads          threads=1
null
> INFO [06-19|14:31:27.284] Commit new mining work          number=242
    sealhash="cf2f3d...b3adca" uncles=0 txs=1 gas=263662 fees=7.90986e-09 elapsed
    =5.425ms
> txpool.status
{
  pending: 1,
  queued: 0
}
```

图 3-4 合约需要打包

这时候需要立刻启动挖矿程序，750s 内挖到一个矿即可以看到回显。

（如果一直测试出问题也可以尝试把 test.sol 中的 say() 函数去掉，重新编译，没有此函数的智能合约时经过反复测试的）

3.2.2 auction.sol

这是本实验主要实现的智能合约，主要的目的是模拟一场简单的竞拍活动。
相关的文件有：

- (1) auction.sol：智能合约的原始文件
- (2) __auction_sol_Auction.bin：编译 auction.sol 生成的 bin 文件

(3) __auction_sol_Auction.abi: 编译 auction.sol 生成的 abi 文件

(4) auction.js: 部署智能合约的文件

相关使用说明和部署方法和 test.sol 相同。这里主要将一下这个智能合约实现的功能。

```
string public sponsor;  
uint public end_time;  
uint256 public value;  
address public spon_addr;  
address public highest_addr;  
uint public part_num;  
uint public highest_num;  
address[] bidders;  
uint256[] bid_value;  
uint256 tmp;  
uint flag;
```

图 3-5 合约变量

主要的变量有：

- (1) sponsor: 交易名称或者发起人名称。
- (2) end_time: 竞拍截止时间。
- (3) value: 竞价最大值或起拍价（无人竞拍时）
- (4) spon_addr: 竞拍发起人地址。
- (5) highest_addr: 竞拍出价最高者地址。
- (6) bidders: 竞拍者地址列表
- (7) bid_value: 竞拍者出价列表
- (8) part_num: 竞拍交易数（每次有人出资都认为是一次交易）
- (9) highest_num: 最高出价的交易号码（号码从 1 开始，每有一次交易则加一）

```
event newspon(string);  
event start_auction(address spon_address,uint et,uint256 s_value);  
event new_higher(address higher_address,uint256 higher_value);  
event end_auction(address spon_address,uint et,uint256 s_value,address  
winner);
```

图 3-6 主要合约事件

主要的事件有：

- (1) newspon: 竞拍更名。主要用于测试。
- (2) start_auction: 发起来一笔交易，会记录发起人地址、截止时间、起拍价。
- (3) new_higher: 有新的高价交易出现，会记录此交易的出资人地址和出资。

(4) **end_auction**: 结束一次交易，会记录下发起人地址、截止时间、拍卖结果价和竞价成功者地址。

主要合约函数有：

(1) **StartAuction**: 发起拍卖函数。此函数默认调用者就是发起者，函数需要拍卖名称、拍卖时间（结束时间=调用函数的时间+拍卖时间）和起拍价，并记录 **start_auction** 事件。

(2) **Bid**: 竞价函数。认为调用者就是竞拍者，并且将 **msg.value** 认为是竞价。竞价函数允许无效竞价，即出价比 **value** 小，这样的交易会被记录，但不会引发 **new_higher** 事件，只有当出价大于 **value** 时会引发 **new_higher** 事件。

此函数也有一些限制，例如只有在截止时间之前才可以竞价。

(3) **EndBid**: 结束竞价函数。只有发起人在截止时间之后可以调用此函数来结束竞拍，当有竞价者参与竞价（如果无人竞价或无人出价大于起拍价，则发起人得不到任何钱），则将最高出价转到发起人的账户，同时产生 **end_auction** 事件。

(4) **Withdraw**: 竞价结束后竞拍者取回资金函数。只有当结束竞价后，竞拍者可以取回自己付出的所有资金。

当然，这其中虽然有很多控制策略，但还是会有一些问题，例如如果在竞拍结束后竞拍者没有及时取回资金，下一笔交易来到后，就取不回来了。

四、实验收获

这门课上，我还是学到了很多东西的。这门课上有很多的工具需要学习、熟悉和应用。

在一开始时，安装和使用 **docker** 就花了我很多时间，在多次使用过 **docker**，才渐渐明白了 **docker** 的一些运行机制，慢慢会做卷映射等工作了。在完成本次施压的过程中，也碰到了非常多的问题，例如有一次突然申请 **web3** 就会报内部错误，这个问题就花了好久才解决，包括后面在部署智能合约的时候，有一段时间挖矿总是超时，花费了大量的时间。还好实验可以顺利完成。

虽然最终实现的合约比较简单，但也是我花了很大心思研究的结果，个人还是比较满意的。